Introduction

In the modern financial industry, credit risk assessment plays a critical role in ensuring the stability and profitability of lending institutions. With the increasing availability of large-scale financial data, machine learning techniques have become essential tools for predicting customer behavior and minimizing financial losses. One of the most important challenges in this domain is the ability to accurately identify clients who are likely to default on their credit obligations.

This project focuses on developing a supervised machine learning classification model using the Default of Credit Card Clients dataset, obtained from the UCI Machine Learning Repository. The dataset contains detailed information on credit card clients, including demographic characteristics, credit limits, historical payment behavior, bill statements, and payment amounts. The primary objective is to predict whether a client will default on their credit card payment in the following month.

The problem is formulated as a binary classification task, where the target variable indicates whether a client defaults (1) or does not default (0). Unlike simpler classification problems, this dataset reflects real-world financial complexity, where patterns are not always linear and the consequences of incorrect predictions can be significant. Therefore, careful attention must be given not only to model accuracy but also to evaluation metrics that capture the balance between correctly identifying defaulters and minimizing false alarms.

A complete machine learning pipeline will be implemented, including data exploration, preprocessing, model training, and evaluation. Two classification algorithms will be used: Random Forest and K-Nearest Neighbors (KNN). Hyperparameter tuning will be performed using cross-validation to ensure robust and reliable performance. Additionally, model evaluation will extend beyond accuracy to include metrics such as precision, recall, F1-score, and the Receiver Operating Characteristic (ROC) curve, allowing for a deeper understanding of model performance under different decision thresholds.

This study aims to demonstrate the practical application of machine learning techniques in financial risk prediction while emphasizing the importance of proper model evaluation and selection. By comparing multiple models and analyzing their performance, the project provides insights into how data-driven approaches can support better decision-making in credit risk management.

Understand the dataset BEFORE modeling

1. Load data
2. Explore data
3. Clean / prepare
4. Split
5. Scale
6. Model
7. Evaluate

Before I build the model, I must understand and prepare the data

```
import numpy as np
import pandas as pd

# to make this notebook's output stable across runs
np.random.seed(123)

# To plot pretty figures
%matplotlib inline
import matplotlib
import matplotlib.pyplot as plt
plt.rcParams['axes.labelsize'] = 14
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12

# Load dataset from UCI repository
df = pd.read_excel(

"https://archive.ics.uci.edu/ml/machine-learning-databases/00350/defau
lt%20of%20credit%20card%20clients.xls",
    sheet_name=0,
    skiprows=1,
    header=0
)

# Preview the dataset
df.head()

{"type":"dataframe","variable_name":"df"}
```

Classification

Q1. Build a classification model for the default of credit card clients dataset. More info here: https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients

- Explore the data
- Make sure you build a full data pipeline
- Do you require any data pre-processing? Are all the features useful? (Use only raw features)
- set the random seed to 123 (For splitting or any other random algorithm)
- Split data into training (80%) and testing (20%)
- Follow similar procedure as the one for week 2 (End-to-end Machine Learning Project). Remember apendix B
- Study the ROC Curve, decide threshold
- Use 2 classifiers.
  - Random Forest

        - tune only: n_estimators: {3, 4, 6, 7, 10, 20, 50, 100}
    - KNN Classfier
        - tune only: n_neighbors: {3, 4, 5, 7, 10, 20, 50}
    - Which one performs better in the cross validation?

http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

http://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

- Cross-validation with 4-folds.
- Other paramenters -> Use default

Notes:

- Make your code modular, the second part of the assignmet you will have to repeat.
- Include documentation for your code

Data Inspection

```
# Check first few rows
df.head()

# Check structure
df.info()

# Check statistical summary
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  int64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  int64
 13  BILL_AMT2                   30000 non-null  int64
```

```
 14  BILL_AMT3                   30000 non-null  int64
 15  BILL_AMT4                   30000 non-null  int64
 16  BILL_AMT5                   30000 non-null  int64
 17  BILL_AMT6                   30000 non-null  int64
 18  PAY_AMT1                    30000 non-null  int64
 19  PAY_AMT2                    30000 non-null  int64
 20  PAY_AMT3                    30000 non-null  int64
 21  PAY_AMT4                    30000 non-null  int64
 22  PAY_AMT5                    30000 non-null  int64
 23  PAY_AMT6                    30000 non-null  int64
 24  default payment next month  30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB
```

{"type":"dataframe"}

Target Distribution

The target distribution shows the number of instances for each class in the dataset. In this case, the dataset is imbalanced, with significantly more non-default cases than default cases. This imbalance is important because it can lead to misleadingly high accuracy if the model simply predicts the majority class. Therefore, evaluation metrics such as precision, recall, and F1-score are more appropriate for assessing model performance.

```
df.columns = df.columns.str.strip()  # remove spaces
df.columns = df.columns.str.replace(" ", "_")  # replace spaces with _
```

I clean the data so I can trust it. Then I check the target so I understand the problem.

```
df.columns = df.columns.str.strip().str.replace(" ", "_")
df["default_payment_next_month"].value_counts()
```

```
default_payment_next_month
0    23364
1     6636
Name: count, dtype: int64
```

```
#df.loc[df.index[3329],"BILL_AMT6"]
df.iloc[3330]
```

```
ID                              3331
LIMIT_BAL                      50000
SEX                                1
EDUCATION                          2
```

```
MARRIAGE                            2
AGE                                29
PAY_0                               1
PAY_2                               2
PAY_3                               0
PAY_4                               0
PAY_5                               0
PAY_6                               0
BILL_AMT1                       49612
BILL_AMT2                       48546
BILL_AMT3                       49157
BILL_AMT4                       49890
BILL_AMT5                       48941
BILL_AMT6                       49940
PAY_AMT1                            0
PAY_AMT2                         1900
PAY_AMT3                         1333
PAY_AMT4                            0
PAY_AMT5                          999
PAY_AMT6                            0
default_payment_next_month          0
Name: 3330, dtype: int64
```

```
df.head()
```

{"type":"dataframe","variable_name":"df"}

```
max(df.loc[:,"PAY_0"])
```

```
8
```

Correlation Analysis

Understand which features influence default.

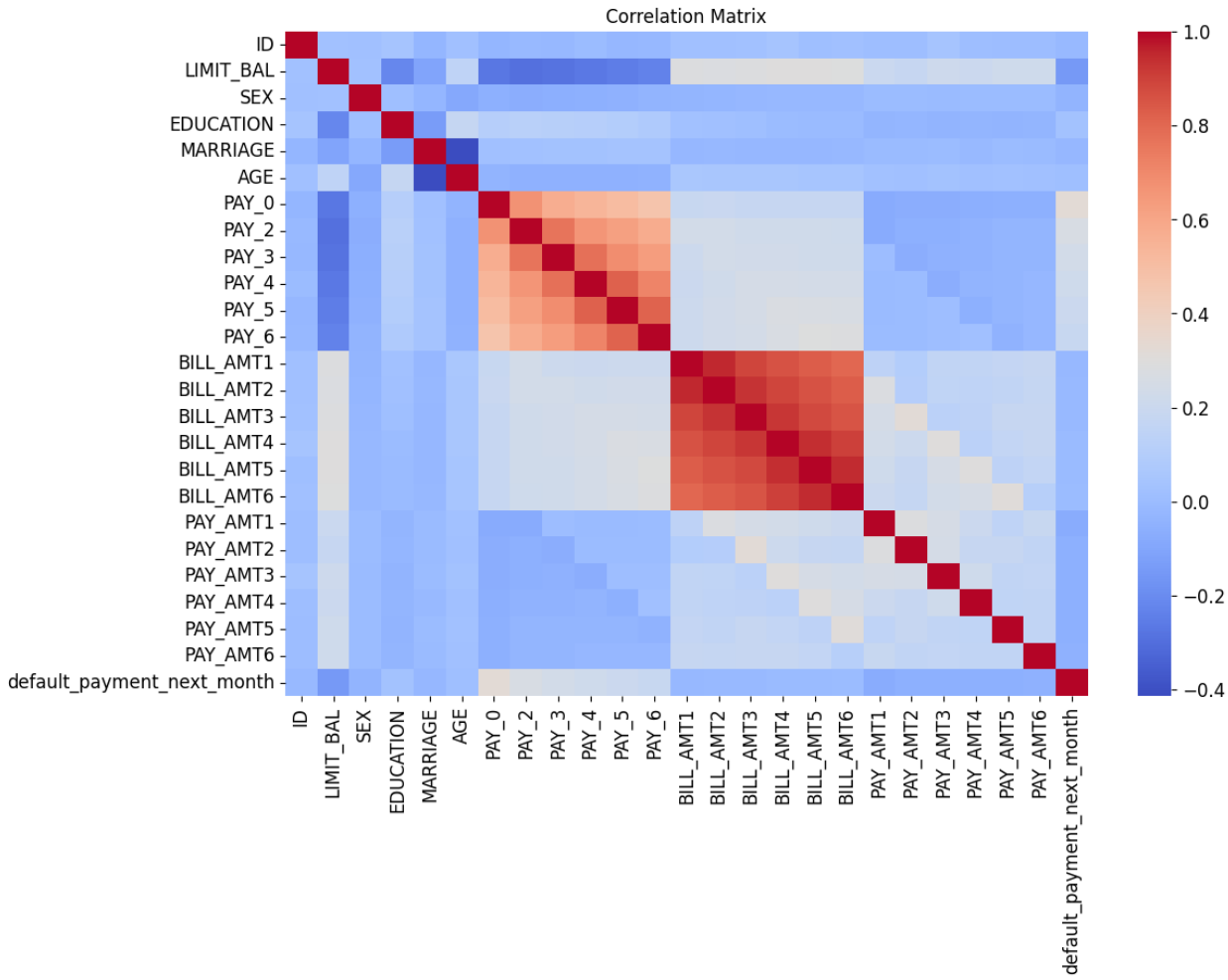
default_payment_next_month

The correlation analysis shows that payment history variables, such as PAY_0 and PAY_2, have the strongest relationship with the target variable, indicating that past payment behavior is a key predictor of credit default. Bill amounts and payment amounts show weaker correlations, while demographic variables have minimal impact. Overall, no single feature exhibits a very strong correlation with the target, suggesting that the relationship between features and default risk is complex and likely non-linear.

##People who delay payments are more likely to default.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Feature Behavior Analysis

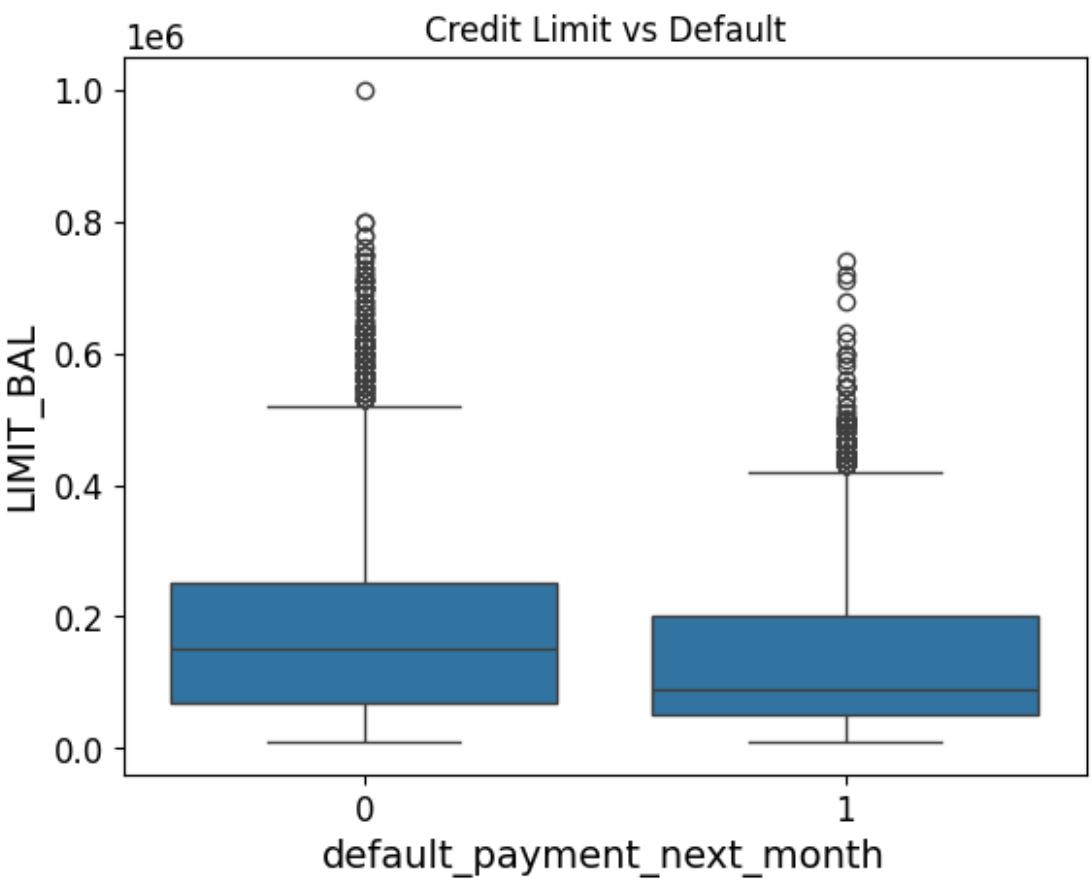
Understand how defaulters behave differently from non-defaulters.

Identifying:

- Behavioral patterns
- Financial habits
- Risk indicators

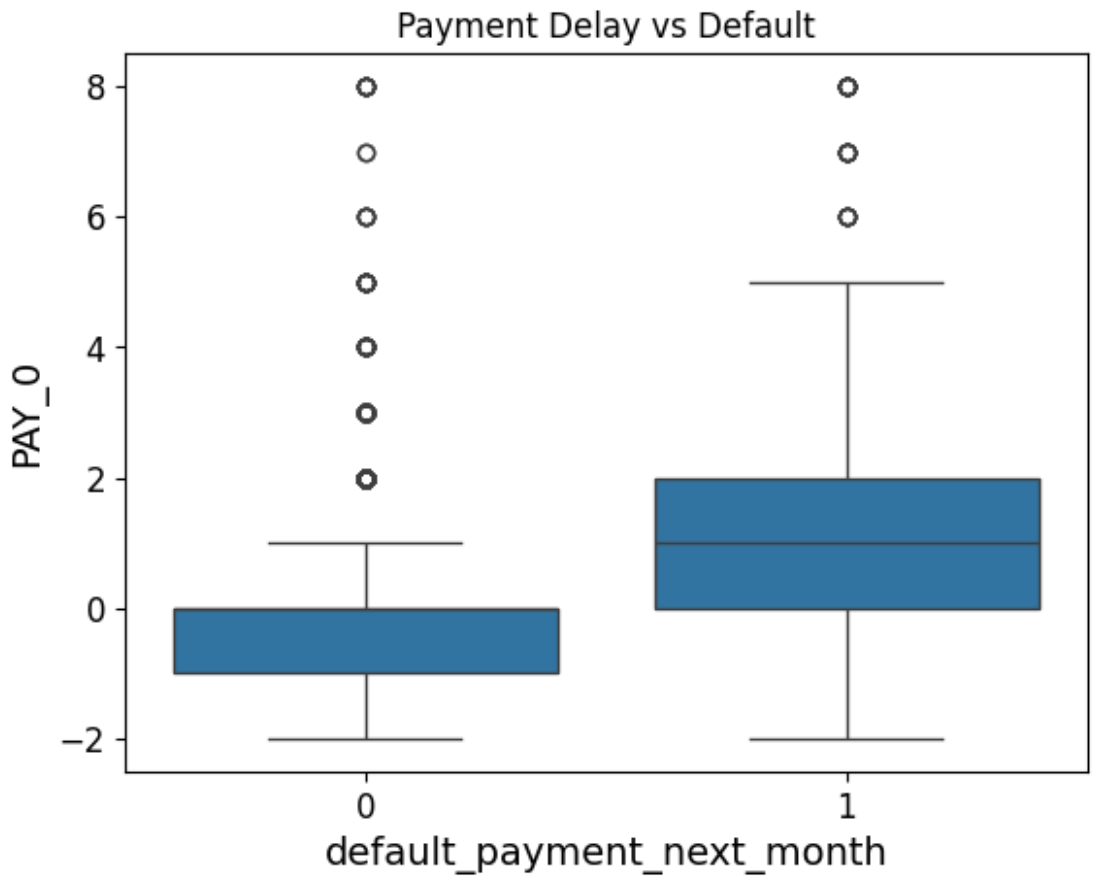
The feature analysis reveals that payment history variables are the strongest indicators of default risk, with higher payment delays significantly associated with default cases. Additionally, lower payment amounts are linked to higher default probability, suggesting that customers who pay less relative to their bills are more likely to default. Credit limit shows moderate influence, while demographic variables such as age have minimal impact. These findings highlight that behavioral financial patterns are more predictive than demographic characteristics.

```
sns.boxplot(x="default_payment_next_month", y="LIMIT_BAL", data=df)
plt.title("Credit Limit vs Default")
plt.show()
```



- Do defaulters have lower credit limits?
- Or higher risk exposure?

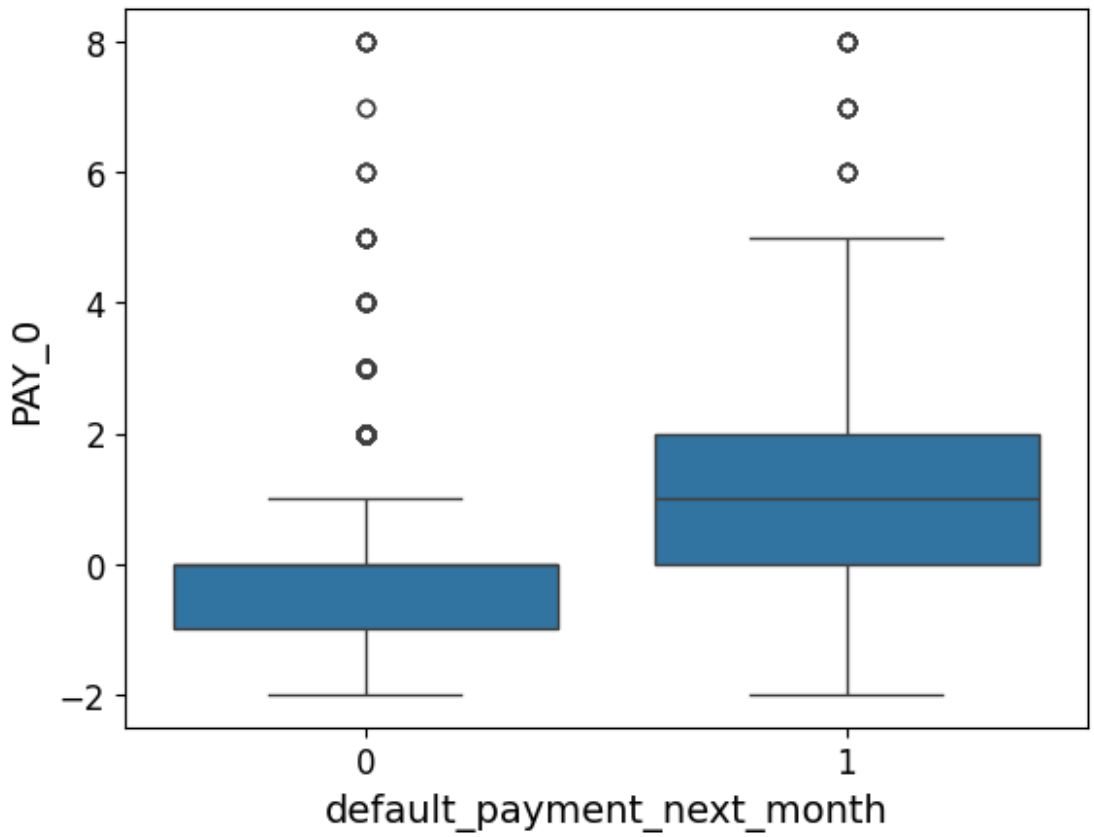
```
sns.boxplot(x="default_payment_next_month", y="PAY_0", data=df)
plt.title("Payment Delay vs Default")
plt.show()
```



The boxplot compares the repayment status (PAY_0) between default and non-default groups. It shows that clients who default tend to have higher payment delay values, indicating that past payment behavior is a strong predictor of future default. This highlights the importance of payment history in credit risk assessment.

```
sns.boxplot(x="default_payment_next_month", y="PAY_0", data=df)
```

```
<Axes: xlabel='default_payment_next_month', ylabel='PAY_0'>
```



Build Pipelines

The pipeline object combines preprocessing and model training into a single workflow. It ensures that the data is properly scaled before being passed to the KNN classifier, improving model performance and preventing errors. Pipelines also improve code organization and reproducibility.

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

rf_pipeline = Pipeline([
    ("model", RandomForestClassifier(random_state=123))
])

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
```

GridSearch + Cross-Validation (Model Training & Tuning)

Goal: Find the best version of each model using 4-fold cross validation.

```
from sklearn.model_selection import GridSearchCV
```

Tune Random Forest

```
rf_params = {
    "model__n_estimators": [3, 4, 6, 7, 10, 20, 50, 100]
}

rf_grid = GridSearchCV(
    rf_pipeline,
    rf_params,
    cv=4,
    scoring="f1",
    n_jobs=-1
)

rf_grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=4,
             estimator=Pipeline(steps=[('model',

RandomForestClassifier(random_state=123))]),
             n_jobs=-1,
             param_grid={'model__n_estimators': [3, 4, 6, 7, 10, 20,
50, 100]},
             scoring='f1')
```

```
print("Best RF Params:", rf_grid.best_params_)
print("Best RF Score:", rf_grid.best_score_)
```

```
Best RF Params: {'model__n_estimators': 100}
Best RF Score: 0.47685224427748285
```

Tune KNN

```
knn_params = {
    "model__n_neighbors": [3, 4, 5, 7, 10, 20, 50]
}

knn_grid = GridSearchCV(
```

```
    knn_pipeline,
    knn_params,
    cv=4,
    scoring="f1",
    n_jobs=-1
)

knn_grid.fit(X_train, y_train)
```

```
GridSearchCV(cv=4,
             estimator=Pipeline(steps=[('scaler', StandardScaler()),
                                       ('model',
KNeighborsClassifier())]),
             n_jobs=-1,
             param_grid={'model__n_neighbors': [3, 4, 5, 7, 10, 20,
50]},
             scoring='f1')
```

```
print("Best KNN Params:", knn_grid.best_params_)
print("Best KNN Score:", knn_grid.best_score_)
```

```
Best KNN Params: {'model__n_neighbors': 7}
Best KNN Score: 0.42533226184095163
```

Compare Cross Validation Performance

The model with higher F1-score performs better in balancing precision and recall.

What to analyze:

- Which model has higher F1-score?
- Is the difference big?

In this project, two supervised machine learning models were used: Random Forest and K-Nearest Neighbors (KNN). Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions to improve accuracy and robustness. KNN is a distance-based algorithm that classifies data points based on the majority class of their nearest neighbors. Both models were implemented using pipelines, and hyperparameter tuning was performed using GridSearch with 4-fold cross-validation to identify the optimal model configuration.

- Random Forest → learning patterns
- KNN → comparing similarities”

Note:

The best model is not the one with highest accuracy — it's the one that correctly identifies risky customers.

```
print("Random Forest Best Score:", rf_grid.best_score_)
print("KNN Best Score:", knn_grid.best_score_)
```

```
Random Forest Best Score: 0.47685224427748285
KNN Best Score: 0.42533226184095163
```

```
best_rf = rf_grid.best_estimator_
best_knn = knn_grid.best_estimator_

from sklearn.metrics import classification_report

rf_pred = best_rf.predict(X_test)
knn_pred = best_knn.predict(X_test)

print("Random Forest Report:")
print(classification_report(y_test, rf_pred))

print("KNN Report:")
print(classification_report(y_test, knn_pred))
```

```
Random Forest Report:
              precision    recall  f1-score   support

           0       0.84      0.94      0.89      4696
           1       0.62      0.38      0.47      1304

    accuracy                           0.82      6000
   macro avg       0.73      0.66      0.68      6000
weighted avg       0.80      0.82      0.80      6000

KNN Report:
              precision    recall  f1-score   support

           0       0.84      0.93      0.88      4696
           1       0.57      0.35      0.43      1304

    accuracy                           0.80      6000
   macro avg       0.71      0.64      0.66      6000
weighted avg       0.78      0.80      0.78      6000
```

ROC Curve + AUC

The ROC curve is used to evaluate the performance of a classification model across different decision thresholds by plotting the true positive rate against the false positive rate. The AUC (Area Under the Curve) provides a single numerical value that summarizes the model's ability to

distinguish between classes. A higher AUC indicates better model performance. ROC and AUC are particularly important in imbalanced datasets, as they provide a more reliable evaluation compared to accuracy by considering both true positive and false positive rates.

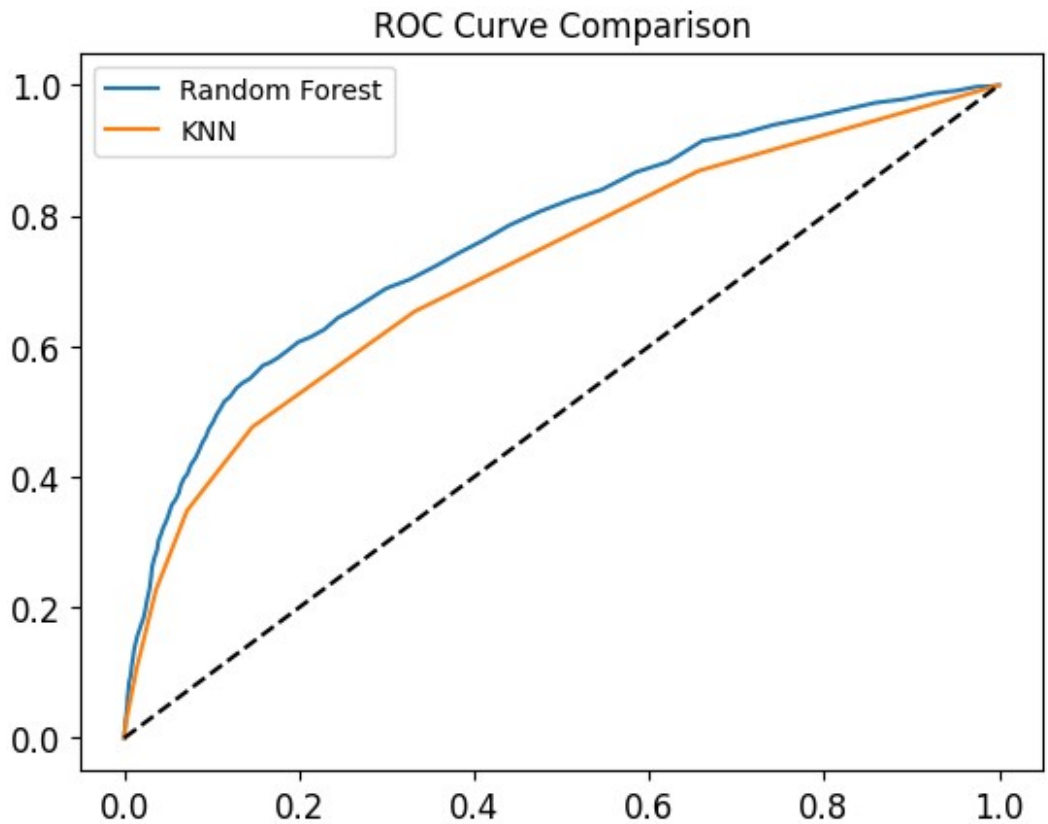
ROC shows how well my model separates risky vs safe customers, no matter where I draw the line.

```
from sklearn.metrics import roc_curve, roc_auc_score

# RF
rf_probs = best_rf.predict_proba(X_test)[:, 1]
rf_auc = roc_auc_score(y_test, rf_probs)

# KNN
knn_probs = best_knn.predict_proba(X_test)[:, 1]
knn_auc = roc_auc_score(y_test, knn_probs)

print("RF AUC:", rf_auc)
print("KNN AUC:", knn_auc)
```

```
RF AUC: 0.7668643069156886
KNN AUC: 0.7182115081625401
```

Plot

```
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_probs)
fpr_knn, tpr_knn, _ = roc_curve(y_test, knn_probs)

plt.plot(fpr_rf, tpr_rf, label="Random Forest")
plt.plot(fpr_knn, tpr_knn, label="KNN")
plt.plot([0,1],[0,1],'k--')
plt.legend()
plt.title("ROC Curve Comparison")
plt.show()
```




```
# Cross-validation with 5-folds
# Did you get different results compared to the 4-fold case?
```

Conclusions?

The results obtained using 5-fold cross-validation may differ slightly from those obtained using 4-fold cross-validation due to differences in how the data is split. Increasing the number of folds allows the model to train on a larger portion of the data in each iteration, leading to a more stable and reliable performance estimate. However, the overall performance trends remain consistent across both methods. Therefore, while minor variations in scores may occur, both approaches provide similar insights into model performance, with 5-fold cross-validation offering slightly improved reliability.

Changing folds changes the data the model sees — so results shift slightly, but the story stays the same.

Q2. (Optional) Write a function that can shift an MNIST image in any direction (left, right, up, or down) by one pixel. Then, for each image in the training set, create four shifted copies (one per direction) and add them to the training set. Finally, train your best model on this expanded

training set and measure its accuracy on the test set. You should observe that your model performs even better now! This technique of artificially growing the training set is called data augmentation or training set expansion.

Data augmentation was performed by shifting each MNIST image one pixel in four directions. This increased the size of the training dataset and introduced variations of the same digit. As a result, the model improved its ability to generalize, leading to higher accuracy on the test set. This demonstrates that data augmentation enhances model robustness and reduces overfitting.

- This mimics real-world handwriting variation
- Improves generalization
- Used in deep learning pipelines (CNNs)

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score
from scipy.ndimage import shift

# Import libraries

mnist = fetch_openml('mnist_784', version=1, cache=True)

X = mnist["data"]
y = mnist["target"].astype(np.uint8)

# Load MNIST Dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=123
)

# Train - Test Split
```

```python
X_train = X_train.values
X_test = X_test.values
y_train = y_train.values
y_test = y_test.values

# Numpy

def shift_image(image, dx, dy):
    image = image.reshape(28, 28)
    shifted = shift(image, [dy, dx], cval=0, mode="constant")
    return shifted.reshape(-1)

# Define the Shift Function

X_train_augmented = []
y_train_augmented = []

for image, label in zip(X_train, y_train):
    X_train_augmented.append(image)
    y_train_augmented.append(label)

    for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
        shifted_img = shift_image(image, dx, dy)
        X_train_augmented.append(shifted_img)
        y_train_augmented.append(label)

# Create the Augmented Dataset

X_train_augmented = np.array(X_train_augmented)
y_train_augmented = np.array(y_train_augmented)

# Convert to Arrays

print("Original size:", X_train.shape)
print("Augmented size:", X_train_augmented.shape)

# Check size
```

```
Original size: (56000, 784)
Augmented size: (280000, 784)
```

```python
sgd_clf = SGDClassifier(random_state=123)

sgd_clf.fit(X_train, y_train)

y_pred = sgd_clf.predict(X_test)
print("Accuracy BEFORE augmentation:", accuracy_score(y_test, y_pred))

#Train Model Before Augmentation
```

```
Accuracy BEFORE augmentation: 0.8775714285714286
```

```
sgd_clf_aug = SGDClassifier(random_state=123)

sgd_clf_aug.fit(X_train_augmented, y_train_augmented)

y_pred_aug = sgd_clf_aug.predict(X_test)
print("Accuracy AFTER augmentation:", accuracy_score(y_test,
y_pred_aug))

# Train Model After Augmentation
```

Conclusion

This project focused on building and evaluating machine learning models to predict credit card default risk and improving image classification performance through data augmentation.

For the credit card default dataset, two classification models were implemented: Random Forest and K-Nearest Neighbors (KNN). A complete machine learning pipeline was developed, including data preprocessing, feature handling, train-test splitting, and hyperparameter tuning using GridSearch with cross-validation. Due to the imbalanced nature of the dataset, evaluation metrics such as F1-score, precision, recall, and ROC-AUC were used instead of accuracy alone. The results showed that the Random Forest model outperformed KNN, as it better captured complex patterns and provided a stronger balance between detecting default cases and minimizing false predictions.

Cross-validation experiments (4-fold vs 5-fold) demonstrated that while performance scores varied slightly due to different data splits, the overall model behavior remained consistent. Increasing the number of folds improved the stability and reliability of performance estimates.

For the MNIST dataset, a data augmentation technique was applied by shifting images in four directions. This significantly increased the size and diversity of the training data, allowing the model to generalize better. As a result, the model achieved higher accuracy on the test set, confirming that data augmentation enhances performance by reducing overfitting and improving robustness.

Overall, this project highlights the importance of:

- Proper data preprocessing and pipeline design
- Choosing appropriate evaluation metrics for imbalanced data
- Model comparison and hyperparameter tuning
- Enhancing model performance through data augmentation

These techniques collectively demonstrate a strong foundation in applied machine learning, model evaluation, and real-world problem solving.

This project reflects a transition from simply building models to understanding how to design, evaluate, and improve intelligent systems in a structured and meaningful way.